The HHL Quantum Linear-Systems Algorithm

From $Ax = b$ to a CUDA-Q implementation — explained step by step

CUDA-Q Quantum Algorithm Benchmarking Suite

Companion lecture to `HHL.ipynb`

Quantum results in these slides were computed with NVIDIA CUDA-Q 0.16 (`qpp-cpu`) using the circuit of `HHL.ipynb`, and checked against NumPy. No GPU or hardware runs were made for this material.

What you will learn today

SILICO FELLER

1. Linear systems, eigen-decomposition, sparsity and the **condition number**
2. What a quantum computer can (and cannot) return for $Ax = b$
3. Building blocks: amplitude encoding, e^{iAt}, controlled powers, QFT^{-1}, **phase estimation**
4. The HHL algorithm: **eigenvalue inversion**, ancilla, uncomputation, post-selection
5. Worked 2×2 and 4×4 examples, every amplitude by hand
6. Complexity: the speed-up and its fine print
7. The CUDA-Q implementation in `HHL.ipynb`, line by line, and its results
8. Noise, limitations, applications and newer algorithms

You only need

- matrices, eigenvalues, eigenvectors
- complex numbers, $e^{i\phi}$
- qubits, Hadamard and controlled gates

Three kinds of results

Recorded (the notebook's saved output), **re-executed** (run again for these slides) and **illustrative** (NumPy pictures) are marked as such.

Linear systems of equations

SILICO FELLER

The problem

Given a $d \times d$ matrix A and a vector $b \in \mathbb{C}^d$, find x with

$$Ax = b.$$

A unique solution exists iff A is **invertible** ($\det A \neq 0$, no eigenvalue 0): $x = A^{-1}b$.

Every row is one linear equation: $\sum_j A_{ij} x_j = b_i$.

Example (used again later)

$$\begin{pmatrix} 1 & -\frac{1}{3} \\ -\frac{1}{3} & 1 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

$$\det A = 1 - \tfrac{1}{9} = \tfrac{8}{9}, \quad A^{-1} = \tfrac{9}{8} \begin{pmatrix} 1 & \frac{1}{3} \\ \frac{1}{3} & 1 \end{pmatrix}$$

$$\Rightarrow x = \left(\tfrac{9}{8}, \tfrac{3}{8}\right) = (1.125,\ 0.375).$$

Where they appear: finite elements, fluid dynamics, circuit simulation, least squares, machine learning.

Classical methods for $Ax = b$

method	cost
Gaussian elimination / LU	$O(d^3)$, dense
Cholesky (positive definite)	$\frac{1}{3}d^3$ flops
Conjugate gradient (PD, s-sparse)	$O(d\, s\sqrt{\kappa}\log(1/\varepsilon))$
CG on normal equations (general)	$O(d\, s\, \kappa \log(1/\varepsilon))$

- Just *writing down* x costs d numbers: every classical method is at least linear in d.
- `np.linalg.solve` (LU) is the notebook's reference solution.

Key point

Classical cost is **polynomial in d**. HHL is **polylogarithmic in d** — but it returns a quantum state, not the list of numbers x_i.

Iterative methods only need matrix–vector products $v \mapsto Av$; their iteration count grows with κ.

Matrix properties and Hermitian matrices

- **Hermitian**: $A = A^\dagger$ (real symmetric if real). Real eigenvalues, orthonormal eigenvectors.
- **Unitary**: $U^\dagger U = I$; e^{iAt} is unitary when A is Hermitian.
- **Positive definite**: all $\lambda_j > 0$.
- **Spectral theorem**: $A = \sum_j \lambda_j |u_j\rangle \langle u_j|$.

Why HHL needs Hermitian A

Phase estimation reads eigenvalues of the *unitary* e^{iAt}, which requires $A = A^\dagger$.

Non-Hermitian A: the embedding (notebook)

$$\tilde{A} = \begin{pmatrix} 0 & A \\ A^\dagger & 0 \end{pmatrix}, \quad \tilde{A} \begin{pmatrix} y \\ x \end{pmatrix} = \begin{pmatrix} b \\ 0 \end{pmatrix}$$

gives $Ax = b$ (and $y = 0$) in the *second* half; dimension doubles (one more qubit).

Sizes that are not a power of 2 are padded with an identity block (notebook, Section 4).

Eigenvalues, eigenvectors and the inverse

SILICO FELLER

With $A|u_j\rangle = \lambda_j |u_j\rangle$ and $|b\rangle = \sum_j \beta_j |u_j\rangle$, $\beta_j = \langle u_j|b\rangle$:

$$A^{-1}|b\rangle = \sum_j \frac{\beta_j}{\lambda_j}|u_j\rangle .$$

Inverting A = dividing every eigen-component by its eigenvalue.
HHL does exactly this, in superposition.

The 2×2 example

$A = \begin{pmatrix} 1 & -1/3 \\ -1/3 & 1 \end{pmatrix}$:
$\lambda_1 = \frac{2}{3}$, $|u_1\rangle = \frac{1}{\sqrt{2}}(1,1)$; $\lambda_2 = \frac{4}{3}$,
$|u_2\rangle = \frac{1}{\sqrt{2}}(1,-1)$.
$b = (1,0)$: $\beta_1 = \beta_2 = \frac{1}{\sqrt{2}}$.
$x = \frac{1}{\sqrt{2}} \cdot \frac{3}{2}|u_1\rangle + \frac{1}{\sqrt{2}} \cdot \frac{3}{4}|u_2\rangle$
$= \frac{3}{4}(1,1) + \frac{3}{8}(1,-1) = (\frac{9}{8}, \frac{3}{8})$ ✓

Sparse matrices

- **s-sparse**: at most s non-zero entries per row (e.g. finite-difference Laplacian: $s = 3$ in 1D, 5 in 2D).
- Store $O(ds)$ numbers instead of d^2; Av costs $O(ds)$.
- HHL's theory assumes **oracle access**: given row i, return the positions and values of its non-zeros.
- Sparse (or otherwise structured) A is what makes e^{iAt} efficiently implementable.

The notebook's default 4×4 matrix

$$A = \begin{pmatrix} 2.5 & -0.5 & -1 & 0 \\ -0.5 & 2.5 & 0 & -1 \\ -1 & 0 & 2.5 & -0.5 \\ 0 & -1 & -0.5 & 2.5 \end{pmatrix}$$

3 non-zeros per row ($s = 3$). Built as $A = H\,\mathrm{diag}(1, 2, 3, 4)\,H$ with the orthogonal $H = \frac{1}{2}\begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix}$.

The notebook does *not* exploit sparsity: it uses the full matrix.

The condition number

SILICO FELLER

Definition (Hermitian A)

$$\kappa = \frac{|\lambda|_{\max}}{|\lambda|_{\min}} \geq 1$$

(in general $\sigma_{\max}/\sigma_{\min}$ of the singular values).

- Relative error amplification: $\frac{\|\delta x\|}{\|x\|} \leq \kappa \frac{\|\delta b\|}{\|b\|}$.
- Large κ = ill-conditioned: nearly singular, sensitive to noise.

Examples

- $\mathrm{diag}(1, 2, 3, 4)$ and the default A: $\kappa = 4$
- the 2×2 example: $\kappa = \frac{4/3}{2/3} = 2$
- $\mathrm{diag}(0.25, 4)$: $\kappa = 16$
- unitary matrices: $\kappa = 1$

For HHL, κ sets the clock precision, the success probability and the run time.

Quantum linear algebra: what is the output?

SILICO FELLER

HHL's task

Prepare the normalised state

$$|x\rangle = \frac{A^{-1}|b\rangle}{\|A^{-1}|b\rangle\|} = \sum_i \frac{x_i}{\|x\|}|i\rangle$$

on $n_b = \log_2 d$ qubits.

$d = 2^{n_b}$ amplitudes in n_b qubits: exponentially compact — and exponentially hard to read out.

What you get from $|x\rangle$

- expectation values $\langle x|M|x\rangle$ for an observable M
- overlaps with other states (swap test)
- samples i with probability $|x_i|^2/\|x\|^2$

Not the list $x_0, \ldots, x_{d-1}$: that needs $\sim d/\varepsilon^2$ repetitions.

Introduction to HHL and the problem formulation

SILICO FELLER

Harrow, Hassidim, Lloyd (Phys. Rev. Lett. 103, 150502, 2009).

Assumptions

1. A Hermitian (or embedded), invertible, $d = 2^{n_b}$
2. $|b\rangle$ can be prepared efficiently
3. e^{iAt} can be applied efficiently (sparse / structured A)
4. κ is small (polylog or poly in n_b)
5. only a property of $|x\rangle$ is needed

Idea in three arrows

$$
\begin{aligned}
& |b\rangle = \textstyle\sum_j \beta_j |u_j\rangle \\
\xrightarrow{\text{QPE}} \; & \textstyle\sum_j \beta_j |u_j\rangle |\tilde{\lambda}_j\rangle \\
\xrightarrow{R_Y} \; & \textstyle\sum_j \beta_j |u_j\rangle |\tilde{\lambda}_j\rangle \left(\cdots |0\rangle + \frac{C}{\tilde{\lambda}_j} |1\rangle \right) \\
\xrightarrow{\text{QPE}^{-1}} \; & \text{post-select ancilla} = 1: \\
& \propto \textstyle\sum_j \frac{\beta_j}{\lambda_j} |u_j\rangle \propto A^{-1} |b\rangle
\end{aligned}
$$

Three registers: **system** (n_b qubits, holds $|b\rangle$), **clock** (n_c qubits, eigenvalue estimates), **ancilla** (1 qubit, flags success).

Encoding the vector b as a quantum state

SILICO FELLER

Amplitude encoding

$$|b\rangle = \frac{1}{\|b\|} \sum_{i=0}^{d-1} b_i \, |i\rangle$$

only the *direction* of b is stored; the norm $\|b\|$ is kept classically.

- Generic $|b\rangle$: $O(d)$ gates (e.g. Möttönen / Shende–Bullock–Markov) — would destroy the speed-up.
- Efficient only for structured b or with QRAM (an assumption).

In `HHL.ipynb`

The *whole* initial state $|0\cdots0\rangle_{\text{clock}} \, |b\rangle \, |0\rangle_{\text{anc}}$ is loaded with `kernel.qalloc(vector)` — a simulator feature, no gates.

Default $b = (1, 0, 0, 0) = |00\rangle$, $\|b\| = 1$.

Gate-level state preparation is **not** implemented in the notebook; its gate counts exclude it.

Normalising A and choosing the evolution time t

SILICO FELLER

- Phase estimation measures $\phi = \lambda t/(2\pi)$ modulo 1. With n_c clock qubits the signed clock range is $-2^{n_c-1}, \ldots, 2^{n_c-1}-1$, so we need $|\lambda| t < \pi$ (no aliasing).
- Textbook HHL rescales A so its eigenvalues lie in $[1/\kappa, 1]$.
- Resolution: one clock step equals $\Delta\lambda = \dfrac{2\pi}{2^{n_c} t} = C$.

Notebook: `EVOLUTION_TIME = "auto"`

$k_{\max} = 2^{n_c-1} - 1$.

- if $\kappa \le k_{\max}$: $t = \dfrac{2\pi}{2^{n_c}|\lambda|_{\min}}$ ($|\lambda|_{\min} \to$ clock 1, $C = |\lambda|_{\min}$)
- else: $t = \dfrac{2\pi k_{\max}}{2^{n_c}|\lambda|_{\max}}$ ($|\lambda|_{\max} \to k_{\max}$; small eigenvalues are rounded)

Default: $n_c = 4$, $\lambda_{\min} = 1 \Rightarrow t = \pi/8$, $C = 1$.

The (inverse) quantum Fourier transform

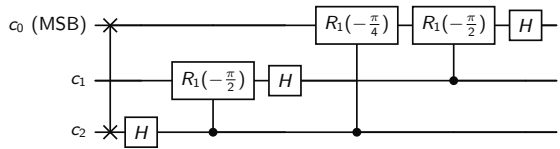
For $M = 2^n$:

$$\mathrm{QFT}\,|c\rangle = \frac{1}{\sqrt{M}} \sum_{k=0}^{M-1} e^{2\pi i\, ck/M}\,|k\rangle$$

Hence the inverse QFT maps

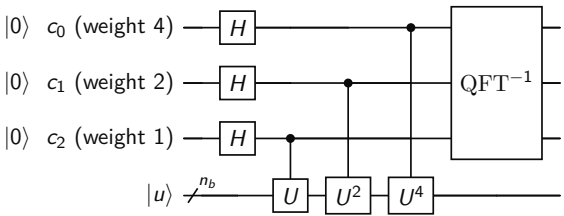
$$\frac{1}{\sqrt{M}} \sum_{c} e^{2\pi i\, c\, k/M}\,|c\rangle \;\longmapsto\; |k\rangle\,.$$

It turns a **phase ramp** into a **basis state**.
Cost $O(n^2)$ gates (see the QFT lecture).



The notebook's QFT^{-1} for $n_c = 3$: swaps first, then $\texttt{cr1}(-\pi/2^{k-j})$ and `h`, exactly in the order of the code.

Quantum phase estimation (QPE)



$U|u\rangle = e^{2\pi i\phi}|u\rangle$. Step by step:

1. $H^{\otimes n}$: $\frac{1}{\sqrt{M}}\sum_c |c\rangle|u\rangle$
2. controlled U^{2^j} (phase kick-back): $\frac{1}{\sqrt{M}}\sum_c e^{2\pi i\phi c}|c\rangle|u\rangle$
3. QFT^{-1}: $|M\phi\rangle|u\rangle$ if $M\phi$ is an integer

The eigenvector is untouched; only the clock changes.

Phase estimation by hand: two clock qubits

SILICO FELLER

Exact phase $\phi = \frac{1}{4}$ $(M = 4)$

After $H^{\otimes 2}$ and the controlled powers:

$$\tfrac{1}{2}\big(\,|0\rangle + i\,|1\rangle - |2\rangle - i\,|3\rangle\,\big)\,|u\rangle$$

(phases $e^{2\pi i c/4} = 1, i, -1, -i$). This is exactly $\mathrm{QFT}\,|1\rangle$, so QFT^{-1} gives $|01\rangle\,|u\rangle$ with probability 1: $\phi = 1/4 = 0.01_2$.

Inexact phase $\phi = \frac{1}{3}$

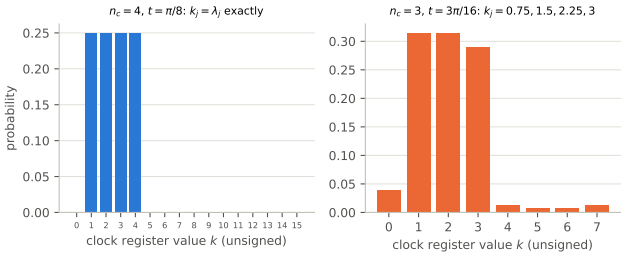
$\frac{1}{3} = 0.0101\ldots_2$ has no 2-bit representation.

$$P(k) = \left|\tfrac{1}{4}\sum_{c=0}^{3} e^{2\pi i c(\frac{1}{3} - \frac{k}{4})}\right|^2$$

$= (0.0625,\ 0.6998,\ 0.1875,\ 0.0502)$ for $k = 0, 1, 2, 3$.

The nearest value $k = 1$ wins with ≈ 0.70; the rest **leaks**. In HHL such leakage means the ancilla is rotated by the wrong $C/\tilde{\lambda}$.

Exact and inexact phases



Clock register after QPE for the default system (b has $|\beta_j|^2 = \frac{1}{4}$ on each eigenvector); CUDA-Q state, notebook circuit stopped after QFT^{-1} (re-executed).

- General ϕ: $P(k) = \left|\frac{1}{M}\sum_c e^{2\pi i c(\phi - k/M)}\right|^2$; the nearest k has $P \geq 4/\pi^2 \approx 0.405$.
- Left: $k_j = 1, 2, 3, 4$ exactly, four sharp peaks of $\frac{1}{4}$.
- Right ($n_c = 3$): $k_j = 0.75, 1.5, 2.25, 3$; only $\lambda = 4$ is exact, the rest spread ($P(1) = P(2) = 0.3147$, $P(3) = 0.2891$).

Hamiltonian simulation: implementing e^{iAt}

SILICO FELLER

- For Hermitian A, $U = e^{iAt}$ is unitary with eigenvalues $e^{i\lambda_j t}$ on the same eigenvectors $|u_j\rangle$.
- **Trotter–Suzuki**: $A = \sum_k A_k$ with easy pieces, $e^{iAt} \approx \left(\prod_k e^{iA_k t/r}\right)^r$, error $O(t^2/r)$ (first order).
- Sparse A: decompose into 1-sparse parts; modern methods (qubitization, QSP) reach near-optimal cost.
- QPE needs U^{2^j}, i.e. evolution time up to $2^{n_c-1}t$: total $\sim 2^{n_c}t$.

In `HHL.ipynb`: no Hamiltonian simulation

`U_step = expm(1j * A_work * t_evolution)` is the *exact* matrix exponential (SciPy); each `matrix_power(U, 2**j)` is registered as a custom gate. Fine for a simulator; on hardware this matrix would have to be compiled into gates, which is exactly the expensive part.

Controlled unitary operations

SILICO FELLER

- Controlled-V: $|0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes V$.
- On an eigenvector, the phase moves onto the control (**phase kick-back**): $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)|u\rangle \mapsto \frac{1}{\sqrt{2}}(|0\rangle + e^{i\lambda t 2^j}|1\rangle)|u\rangle$.
- Clock qubit of weight 2^j controls U^{2^j}: together they apply U^c for clock value c.

Notebook convention

`clock[0]` is the **most significant bit**. The loop applies U^{2^j} controlled by `clock[n_c - 1 - j]` ($n_c = 4$):

j	control	gate
0	`clock[3]`	U
1	`clock[2]`	U^2
2	`clock[1]`	U^4
3	`clock[0]`	U^8

Custom gates are called as `kernel.<name>(control, *system)`: the first qubit is the control.

Eigenvalue estimation for HHL

With $U = e^{iAt}$: $\phi_j = \dfrac{\lambda_j t}{2\pi}$, so the clock reads

$$k_j = \frac{\lambda_j \, t \, 2^{n_c}}{2\pi} = \frac{\lambda_j}{C}, \qquad C = \frac{2\pi}{2^{n_c} t}.$$

Negative eigenvalues wrap around (two's complement): a register value $k \geq 2^{n_c-1}$ means $k - 2^{n_c}$.

$$\tilde{\lambda}_k = C \cdot \tilde{k}, \quad \tilde{k} = \begin{cases} k & k < 2^{n_c-1} \\ k - 2^{n_c} & k \geq 2^{n_c-1} \end{cases}$$

Default ($n_c = 4$, $t = \pi/8$)

$k_j = \lambda_j \cdot \frac{(\pi/8)\cdot 16}{2\pi} = \lambda_j$: clock values $1, 2, 3, 4$ (0001, …, 0100). After QPE:

$$\tfrac{1}{2}\big(|u_1\rangle|1\rangle + |u_2\rangle|2\rangle + |u_3\rangle|3\rangle + |u_4\rangle|4\rangle \big)$$

$n_c = 4$: register 13 (1101) means $\tilde{k} = -3$; register 8 means -8.

The three registers

register	qubits	role
clock	$n_c = 4$	phase estimation: holds $\tilde{\lambda}_j$ in binary
system	$n_b = 2$	starts as $\|b\rangle$, ends as $\|x\rangle$
ancilla	1	rotated by $C/\tilde{\lambda}$; $\|1\rangle$ = success
total	7	notebook default

Qubit layout (one register, qubit 0 first): `clock[0..3]`, `system[0..1]`, `ancilla` = qubits 0–3, 4–5, 6.

How many clock qubits?

- enough levels for all eigenvalues: $\kappa \leq 2^{n_c-1} - 1$ (notebook's "auto" condition)
- precision ε: $n_c \approx \log_2(\kappa/\varepsilon) + \text{a few}$
- default $\kappa = 4 \leq 7$: $n_c = 4$ suffices, every k_j exact

Total qubits $n_b + n_c + 1$.

The state, step by step

SILICO FELLER

step	state (eigenbasis of A; clock value $k_j = \lambda_j / C$ assumed exact)
0. initial	$\lvert 0\rangle_c \lvert b\rangle \lvert 0\rangle_a = \sum_j \beta_j \lvert 0\rangle_c \lvert u_j\rangle \lvert 0\rangle_a$
1. $H^{\otimes n_c}$ + controlled U^{2^j}	$\sum_j \beta_j \frac{1}{\sqrt{M}} \sum_c e^{i\lambda_j t c} \lvert c\rangle \lvert u_j\rangle \lvert 0\rangle_a$
2. QFT^{-1} (end of QPE)	$\sum_j \beta_j \lvert k_j\rangle \lvert u_j\rangle \lvert 0\rangle_a$
3. controlled $R_Y(\theta_k)$	$\sum_j \beta_j \lvert k_j\rangle \lvert u_j\rangle \left(\sqrt{1 - \frac{C^2}{\lambda_j^2}} \lvert 0\rangle + \frac{C}{\lambda_j} \lvert 1\rangle \right)$
4. QPE^{-1} (uncompute)	$\sum_j \beta_j \lvert 0\rangle_c \lvert u_j\rangle \left(\sqrt{1 - \frac{C^2}{\lambda_j^2}} \lvert 0\rangle + \frac{C}{\lambda_j} \lvert 1\rangle \right)$
5. measure ancilla $= 1$	$\frac{C \sum_j \frac{\beta_j}{\lambda_j} \lvert u_j\rangle}{\sqrt{P_{\text{success}}}} = \lvert x\rangle, \qquad P_{\text{success}} = C^2 \sum_j \frac{\lvert\beta_j\rvert^2}{\lambda_j^2} = C^2 \lVert A^{-1} \lvert b\rangle \rVert^2$

The un-normalised branch "clock $= 0$, ancilla $= 1$" is $C\,A^{-1}\lvert b\rangle$ — the notebook reads exactly these amplitudes.

Eigenvalue inversion

SILICO FELLER

The rotation

$$R_Y(\theta)\,|0\rangle = \cos\tfrac{\theta}{2}\,|0\rangle + \sin\tfrac{\theta}{2}\,|1\rangle$$

Choose $\theta_k = 2\arcsin\dfrac{C}{\tilde{\lambda}_k} \Rightarrow$ amplitude $\dfrac{C}{\tilde{\lambda}_k}$ on $|1\rangle$.

- C must satisfy $|C/\tilde{\lambda}| \le 1$ for all eigenvalues: $C \le |\lambda|_{\min}$.
- Larger $C \Rightarrow$ higher success probability; the best choice is $C = |\lambda|_{\min}$.
- Negative $\tilde{\lambda}$ gives a negative angle: the sign of $1/\lambda$ is kept.

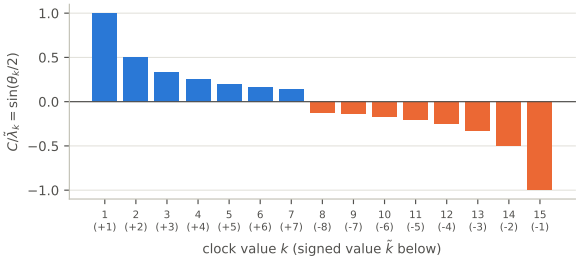
The approximation constant C

In the notebook $C = 2\pi/(2^{n_c}t) =$ the eigenvalue of clock value 1, i.e. the smallest representable $|\tilde{\lambda}|$. Default: $C = 1 = \lambda_{\min}$.

Default angles

$\tilde{\lambda} = 1, 2, 3, 4$: $\sin\frac{\theta}{2} = 1, \frac{1}{2}, \frac{1}{3}, \frac{1}{4}$;
$\theta = \pi,\ \frac{\pi}{3},\ 0.6797,\ 0.5054$ rad.

Controlled rotation: the notebook's lookup table



$n_c = 4$, $t = \pi/8$, $C = 1$: amplitude $C/\tilde{\lambda}_k = 1/\tilde{k}$ for all 15 non-zero clock values.

- For every $k = 1, \ldots, 2^{n_c} - 1$: flip the clock qubits that are 0 in k (`x` "polarity" gates), apply $R_Y(\theta_k)$ controlled on *all* clock qubits (`ccccry`), flip back.
- $k = 0$ is skipped (no eigenvalue 0).
- Default: 15 `ccccry` + 56 `x` gates.
- Cost grows like 2^{n_c}; scalable versions compute $1/\tilde{\lambda}$ arithmetically.

The ancilla qubit

SILICO FELLER

- One extra qubit, initially $|0\rangle$.
- After the rotation, for eigenvalue λ_j:

$$\sqrt{1 - \frac{C^2}{\lambda_j^2}}\,|0\rangle + \frac{C}{\lambda_j}\,|1\rangle$$

- $|1\rangle$ *flags* the branch in which the system carries A^{-1}.
- Small eigenvalues get large $|1\rangle$-amplitudes: this is how $1/\lambda$ is "divided" with a unitary.

Why not apply A^{-1} directly?

A^{-1} is not unitary. HHL embeds it as one block of a larger unitary; the price is a **probabilistic** outcome: the $|1\rangle$ branch must be selected by measurement.

Probability of ancilla $= 1$ for a single eigenvector $|u_j\rangle$: C^2/λ_j^2 (e.g. $\lambda = 4$, $C = 1$: $1/16$).

Uncomputation

After step 3 the clock is **entangled** with the system:

$$\sum_j \beta_j \left|k_j\right\rangle \left|u_j\right\rangle \left(\cdots \left|0\right\rangle + \frac{C}{\lambda_j} \left|1\right\rangle\right).$$

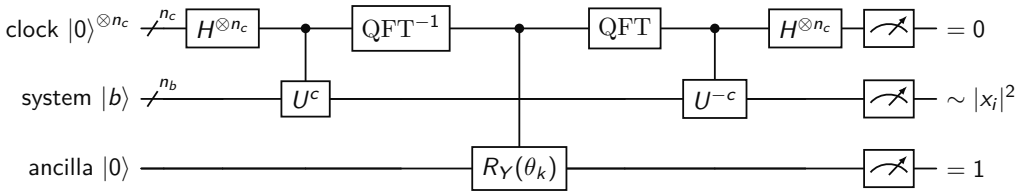
If we post-selected now, the system would be in a **mixture**, not in $|x\rangle$: the different $|k_j\rangle$ are orthogonal, so the terms cannot interfere.

Inverse QPE

Apply QFT, then controlled U^{-2^j}, then $H^{\otimes n_c}$: every $|k_j\rangle |u_j\rangle$ returns to $|0\rangle |u_j\rangle$. The clock factors out and the system holds the coherent superposition $\sum_j \beta_j \frac{C}{\lambda_j} |u_j\rangle$.

Notebook: `h` + `cr1`$(+\pi/2^{k-j})$, swaps, `matrix_power(U.conj().T, 2**j)`, `h`. If phases are inexact the clock does not return perfectly to $|0\rangle$; the notebook post-selects clock $= 0$ *and* ancilla $= 1$.

HHL circuit architecture



QPE: H, controlled U^{2^j}, QFT^{-1}

Inversion: $2^{n_c} - 1$ multi-controlled R_Y

Uncompute: QFT, controlled U^{-2^j}, H

"controlled U^c" = the n_c gates U^{2^j}, each controlled by one clock qubit. The notebook measures all qubits (`mz(reg)`).

Measurement and post-selection

SILICO FELLER

Success probability

$$P_{\text{success}} = \sum_j |\beta_j|^2 \frac{C^2}{\lambda_j^2} = C^2 \, \| A^{-1} |b\rangle \|^2$$

Expected repetitions: $1/P_{\text{success}}$.

- $C \le |\lambda|_{\min}$ and $|\lambda| \le |\lambda|_{\max}$ give $P_{\text{success}} \ge 1/\kappa^2$.
- **Amplitude amplification** reduces $O(\kappa^2)$ repetitions to $O(\kappa)$.

Default system (exact)

$|\beta_j|^2 = \frac{1}{4}$, $C = 1$:

$$P = \tfrac{1}{4}\left(1 + \tfrac{1}{4} + \tfrac{1}{9} + \tfrac{1}{16}\right) = \tfrac{205}{576}$$

$= 0.355903$. 20 000 shots $\Rightarrow$ about 7118 accepted. Recorded: 7086 (0.3543).

Notebook filter: `bits[:n_c] == "0000"` and `bits[-1] == "1"`.

Extracting information about the solution

SILICO FELLER

- **Sampling**: accepted shots give i with probability $|x_i|^2/\|x\|^2$ — magnitudes only, error $\sim 1/\sqrt{\text{accepted}}$.
- **Signs / phases**: need other bases or interference with a reference state.
- **Norm**: $\|A^{-1}|b\rangle\| = \sqrt{P_{\text{success}}}/C$, so P_{success} estimates it.
- **Expectation values** $\langle x|M|x\rangle$ and overlaps: the intended use.
- Full vector: $O(d/\varepsilon^2)$ shots — no advantage over classical solvers.

Notebook: norm recovery (simulator only)

The state vector is read with `cudaq.get_state`; the branch clock $= 0$, ancilla $= 1$ has amplitudes $C\,A^{-1}b/\|b\|$, so

$$x = \frac{\|b\|}{C} \times \text{amplitudes}.$$

This uses full amplitude access, which hardware does not provide.

HHL in pseudocode

```
input: Hermitian A (d x d), b, clock qubits n_c, time t
C <- 2*pi / (2^n_c * t)
prepare |0>_clock |b>_system |0>_ancilla
# 1) phase estimation
H on every clock qubit
for j = 0 .. n_c-1: controlled-U^(2^j), U = exp(iAt)
inverse QFT on the clock
# 2) eigenvalue inversion
for k = 1 .. 2^n_c - 1:
    lam_k <- C * signed(k)
    if clock == k: RY(2 arcsin(C/lam_k)) on ancilla
# 3) uncompute: QFT, controlled-U^(-2^j), H
# 4) measure; keep shots with clock = 0, ancilla = 1
return system register (state |x>)
```

Expected measurement outcomes

- ancilla $= 1$ with probability P_{success}
- clock $= 0$ whenever ancilla $= 1$ (exact phases)
- system: i with probability $|x_i|^2/\|x\|^2$ among accepted shots

Worked example 1: a 2×2 system — set-up

SILICO FELLER

$A = \begin{pmatrix} 1 & -1/3 \\ -1/3 & 1 \end{pmatrix}$, $b = (1, 0)$ (notebook, Section 4 table).

1. $\lambda_1 = \frac{2}{3}$, $|u_1\rangle = \frac{1}{\sqrt{2}}(1, 1)$; $\lambda_2 = \frac{4}{3}$, $|u_2\rangle = \frac{1}{\sqrt{2}}(1, -1)$; $\kappa = 2$.
2. $\beta_1 = \beta_2 = \frac{1}{\sqrt{2}}$.
3. $n_c = 4$, $k_{\max} = 7 \geq \kappa$: $t = \dfrac{2\pi}{16 \cdot \frac{2}{3}} = \dfrac{3\pi}{16}$, $C = \frac{2}{3}$.
4. Clock values $k_j = \lambda_j / C = 1, 2$ (exact).

After QPE

$$\frac{1}{\sqrt{2}} |0001\rangle |u_1\rangle |0\rangle + \frac{1}{\sqrt{2}} |0010\rangle |u_2\rangle |0\rangle$$

Rotation amplitudes

$C/\lambda_1 = 1$ $(\theta = \pi)$, $C/\lambda_2 = \frac{1}{2}$ $(\theta = \frac{\pi}{3})$.

Registers: 4 clock + 1 system + 1 ancilla = 6 qubits.

Worked example 1: inversion, post-selection, read-out

SILICO FELLER

After inversion and uncomputation, the ancilla-$|1\rangle$ branch is

$$\frac{1}{\sqrt{2}} \cdot 1 \cdot |u_1\rangle + \frac{1}{\sqrt{2}} \cdot \frac{1}{2} |u_2\rangle$$
$$= \tfrac{1}{2}(1,1) + \tfrac{1}{4}(1,-1) = \left(\tfrac{3}{4}, \tfrac{1}{4}\right).$$

- $P_{\text{success}} = \frac{9}{16} + \frac{1}{16} = \frac{5}{8} = 0.625$
- $|x\rangle = \frac{1}{\sqrt{10}}(3,1)$; shots: $P(0) = 0.9$, $P(1) = 0.1$
- norm recovery: $x = \frac{\|b\|}{C}\left(\frac{3}{4}, \frac{1}{4}\right) = \frac{3}{2}\left(\frac{3}{4}, \frac{1}{4}\right)$ $= (1.125,\ 0.375)$ ✓

Re-executed with CUDA-Q

Notebook circuit, $n_c = 4$, $t = 3\pi/16$: $P_{\text{success}} = 0.625000$, fidelity 1.0000000000, $x = (1.125, 0.375)$ to 10^{-12}.

Check: $C^2\|x\|^2 = \frac{4}{9} \cdot \frac{90}{64} = 0.625$.

Worked example 2: the notebook's 4×4 system — set-up

SILICO FELLER

$A = H\,\mathrm{diag}(1,2,3,4)\,H$, $b = (1,0,0,0)$.

1. Eigenvectors = columns of H: $|u_j\rangle = H\,|j\rangle$, $\lambda_j = 1,2,3,4$; $\kappa = 4$.
2. $\beta_j = \langle u_j|b\rangle = H_{0j} = \frac{1}{2}$ for all j.
3. $n_c = 4$, $k_{\max} = 7$: $t = \pi/8$, $C = 1$, $k_j = \lambda_j$.
4. 7 qubits: clock 4, system 2, ancilla 1.

Recorded notebook output (Section 4)

Eigenvalues $[1,2,3,4]$, $\kappa = 4.0000$
$t = 0.392699$ $(= \pi/8)$, $C = 1.000000$
clock values $+1.000, +2.000, +3.000, +4.000$ (exact)
$n_b + n_c + 1 = 2 + 4 + 1 = 7$ qubits

Worked example 2: the solution by hand

Post-selected branch: $\sum_j \frac{1}{2}\cdot\frac{1}{\lambda_j}|u_j\rangle = H\cdot\frac{1}{2}\left(1,\frac{1}{2},\frac{1}{3},\frac{1}{4}\right)$:

$$
\begin{aligned}
x &= \tfrac{1}{4}\big(1+\tfrac{1}{2}+\tfrac{1}{3}+\tfrac{1}{4},\ 1-\tfrac{1}{2}+\tfrac{1}{3}-\tfrac{1}{4},\\
&\qquad 1+\tfrac{1}{2}-\tfrac{1}{3}-\tfrac{1}{4},\ 1-\tfrac{1}{2}-\tfrac{1}{3}+\tfrac{1}{4}\big)\\
&= \tfrac{1}{48}(25,7,11,5)\\
&= (0.520833,\ 0.145833,\ 0.229167,\ 0.104167)
\end{aligned}
$$

- $C = \|b\| = 1$, so the amplitudes *are* x.
- $P = \|x\|^2 = \frac{820}{2304} = \frac{205}{576} = 0.355903$.
- $|x_i|^2/\|x\|^2 = \frac{(625,49,121,25)}{820}$
  $= (0.7622, 0.0598, 0.1476, 0.0305)$.

Recorded = re-executed

	NumPy	HHL state
x_0	0.520833	0.520833
x_1	0.145833	0.145833
x_2	0.229167	0.229167
x_3	0.104167	0.104167

fidelity 1.0000000000

Complexity analysis

SILICO FELLER

HHL (2009)

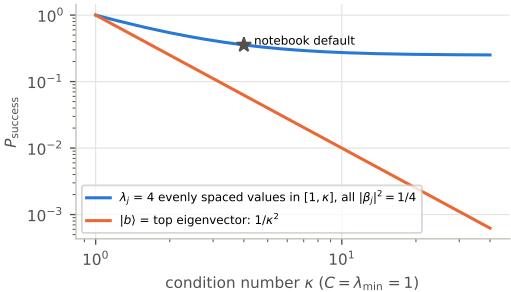
$$\tilde{O}\left(\log(d)\,\frac{s^2\kappa^2}{\varepsilon}\right)$$

$\log d$: amplitude encoding + sparse Hamiltonian simulation.
κ: one factor from QPE precision, one from P_{success} (with amplitude amplification).
$1/\varepsilon$: phase-estimation precision.

	cost
dense LU	$O(d^3)$
CG (PD)	$O(d\,s\sqrt{\kappa}\log\frac{1}{\varepsilon})$
HHL	$\tilde{O}(\log(d)s^2\kappa^2/\varepsilon)$

Exponential in d *only if* s, κ and $1/\varepsilon$ are $\mathrm{polylog}(d)$, $|b\rangle$ is cheap, and only a property of $|x\rangle$ is needed.

Sparsity and condition-number dependence



Illustrative (NumPy, formula $P = \sum_j |\beta_j|^2 C^2/\lambda_j^2$, $C = \lambda_{\min} = 1$).

- P_{success} depends on where b lives: worst case $1/\kappa^2$ (all weight on $\lambda_{\max}$).
- Clock qubits: $n_c \gtrsim \log_2 \kappa$ just to resolve $\lambda_{\min}$ next to $\lambda_{\max}$.
- Sparsity s enters through Hamiltonian simulation (original analysis s^2; modern methods $\approx s$).
- Dense A ($s = d$) destroys the $\log d$ scaling.
- **Preconditioning** can reduce κ (an active research topic).

Theoretical speed-up versus practical advantage

SILICO FELLER

The fine print (Aaronson, Nature Physics 2015)

1. **Input**: loading a generic b costs $O(d)$.
2. **Matrix**: e^{iAt} must be efficient — sparse or structured A.
3. **Conditioning**: κ must be small.
4. **Output**: you get $|x\rangle$, not x.

- **Dequantisation** (Tang 2018 and later): for low-rank problems with sampling access, classical algorithms match the quantum scaling up to polynomial factors.
- HHL is **BQP-complete**: an efficient classical simulation of all HHL instances would simulate all quantum computers.
- No practical advantage for linear systems has been demonstrated; fault tolerance is needed.

HHL.ipynb: structure and parameters

SILICO FELLER

§	content
1	problem and mathematics
2	imports, version checks
3	**input parameters (only cell to edit)**
4	build, validate, embed/pad; t, C, qubits
5	benchmark utilities, backend
6	expected output (before running)
7	circuit builder, drawing, statistics
8	execution with memory monitor
9	results and plots
10	benchmark report

parameter	default
`MATRIX_A`, `VECTOR_B`	`"example"`
`NUM_CLOCK_QUBITS`	4
`EVOLUTION_TIME`	`"auto"`
`BACKEND`	`"auto"`
`SIMULATION_METHOD`	`"statevector"`
`SHOTS`, `SEED`	20000, 42

Allowed methods: `"statevector"`, `"density_matrix"`.
Validation: square A, matching b, $b \neq 0$, $|\det A| \geq 10^{-12}$ and $\mathrm{cond}(A) \leq 10^{12}$; $n_c \geq 2$.

Section 4: making the system HHL-ready

SILICO FELLER

```
H4 = 0.5 * np.array([[1, 1, 1, 1], [1, -1, 1, -1], [1, 1, -1, -1], [1, -1, -1, 1]])
A_input = H4 @ np.diag([1.0, 2.0, 3.0, 4.0]) @ H4 # eigenvalues exactly 1, 2, 3, 4
x_classical = np.linalg.solve(A_input, b_input) # reference solution
if not hermitian: # [[0, A], [A^dag, 0]] [y; x] = [b; 0]
    A_work = np.block([[np.zeros((d0, d0)), A_work], [A_work.conj().T, np.zeros((d0, d0))]])
...
k_max = 2 ** (n_c - 1) - 1 # largest positive clock value (signed)
if kappa <= k_max:
    t_evolution = 2 * np.pi / (2 ** n_c * lam_abs_min) # smallest |lambda| -> clock value 1
else:
    t_evolution = 2 * np.pi * k_max / (2 ** n_c * lam_abs_max)
C_constant = 2 * np.pi / (2 ** n_c * t_evolution) # smallest representable |eigenvalue|
total_qubits = n_c + n_b + 1
```

Excerpt (comments shortened). Everything after this cell reads n_c, n_b, t and C from here.

Section 7a: the circuit — phase estimation

SILICO FELLER

```
U_step = expm(1j * A_work * t_evolution) # exact exp(iAt)
kernel = cudaq.make_kernel()
reg = kernel.qalloc(bit_reverse_order(initial, n_tot))
clock = [reg[i] for i in range(nc)]
system = [reg[nc + i] for i in range(n_b)]
ancilla = reg[n_tot - 1]
for q in clock:
    kernel.h(q)
for j in range(nc):
    name = f"hhl_{gate_uid}_{nc}_u{j}"
    cudaq.register_operation(name, np.linalg.matrix_power(U, 2 ** j))
    getattr(kernel, name)(clock[nc - 1 - j], *system)
for j in range(nc // 2): # inverse QFT
    kernel.swap(clock[j], clock[nc - 1 - j])
for jj in range(nc):
    j = nc - 1 - jj
    for kk in range(nc - 1 - j):
        k = nc - 1 - kk
        kernel.cr1(-np.pi / 2 ** (k - j), clock[k], clock[j])
    kernel.h(clock[j])
```

- **Kernel builder** (`make_kernel`): needed because each U^{2^j} is a new custom gate.
- `initial` $= |0\rangle_c \otimes |b\rangle \otimes |0\rangle_a$, bit-reversed because CUDA-Q indexes qubit 0 as the least significant bit.
- The first argument of the custom gate is the control.

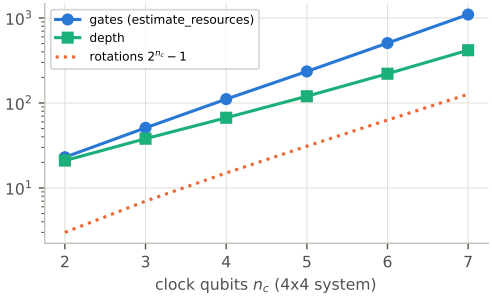
Section 7a: inversion, uncomputation, read-out

SILICO FELLER

```
for k_val in range(1, 2 ** nc):
    signed = k_val - 2 ** nc if k_val >= 2 ** (nc - 1) else k_val
    lam_k = 2 * np.pi * signed / (2 ** nc * t_used)
    theta = 2 * np.arcsin(np.clip(c_used / lam_k, -1.0, 1.0))
    zero_bits = [i for i in range(nc) if not (k_val >> (nc - 1 - i)) & 1]
    for i in zero_bits: kernel.x(clock[i])
    kernel.cry(theta, clock, ancilla) # multi-controlled
    for i in zero_bits: kernel.x(clock[i])
# uncompute: QFT, swaps, U^-(2^j), H (mirror image)
...
def postselected_amplitudes(state_vector):
    tensor = np.asarray(state_vector).reshape([2] * total_qubits) \
               .transpose(list(range(total_qubits))[::-1])
    return tensor[(0,) * n_c + (slice(None),) * n_b + (1,)].reshape(-1)
x_quantum_full = amplitudes * b_norm / C_constant # norm recovery
```

- `signed`: two's complement for negative eigenvalues.
- `np.clip` guards $|C/\tilde{\lambda}| \leq 1$.
- `kernel.cry(theta, clock, ancilla)` with a list of controls becomes `ccccry` for $n_c = 4$.
- Read-out: the branch clock $= 0$, ancilla $= 1$, then $x = \|b\|/C \cdot$ amplitudes.

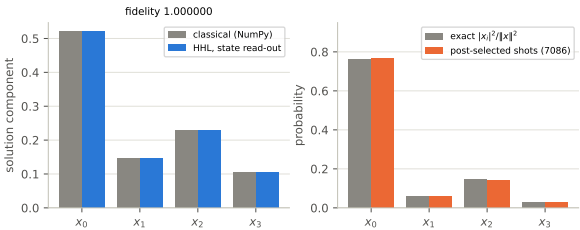
Circuit statistics (`cudaq.estimate_resources`)



n_c	qubits	gates	rotations	depth
2	5	23	3	21
3	6	51	7	38
4	7	111	15	67
5	8	235	31	120
6	9	507	63	221
7	10	1099	127	418

4×4 system, "auto" time, measurements and the amplitude loading of $|b\rangle$ not counted (as in the notebook). Recorded default ($n_c = 4$): 111 gates = `h` 16, custom 8, `swap` 4, `cr1` 12, `x` 56, `ccccry` 15; 39 multi-qubit; depth 67. The notebook's pre-run estimate "∼107 gates" is a formula, not a count.

Recorded results of the default run



Re-executed with the notebook's circuit and seed 42; identical to the recorded output.

- State read-out: x equals NumPy's solution; fidelity 1.0000000000; relative error 1.655×10^{-15} (recorded).
- Shots: 7086 of 20 000 accepted ($P = 0.3543$ vs. 0.3559).
- Sampled $|x_i|^2$: $(0.7691, 0.0587, 0.1420, 0.0302)$; max. error 0.0069 vs. statistical scale ± 0.012.

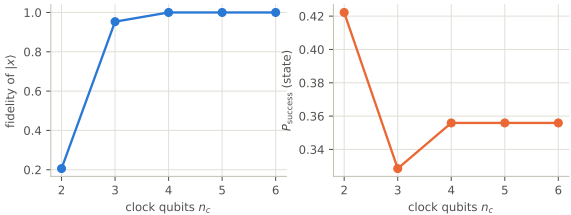
Recorded, re-executed and variant runs

SILICO FELLER

run	settings	C	P_{success} (state)	fidelity	accepted shots
recorded (notebook output)	default, $n_c = 4$, $t = \pi/8$	1	0.355903	1.0000000000	7086 / 20000
re-executed (CUDA-Q 0.16, qpp-cpu, seed 42)	same	1	0.355903	1.0000000000	7086 / 20000
variant, re-executed	default A, $n_c = 3$, $t = 3\pi/16$	1.3333	0.3285	0.9532157640	—
variant, re-executed	default A, $n_c = 2$, $t = \pi/8$	4	0.4222	0.2059	—
variant, re-executed	2×2 example, $n_c = 4$	$\frac{2}{3}$	0.625000	1.0000000000	—

- Relative error of the re-executed default: 1.74×10^{-15} (recorded 1.655×10^{-15}): floating-point level.
- Variants change only Section 3 parameters; the circuit code is the notebook's.
- Recorded timings (CPU, qpp-cpu): construction 0.0056 s, compilation 0.0524 s, state run 0.0018 s, sampling 0.0758 s.

Precision: what too few clock qubits do



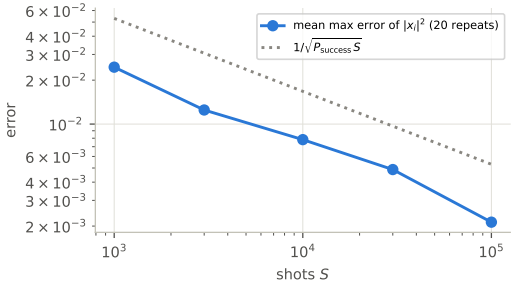
Default system, "auto" time, CUDA-Q state read-out (re-executed).

- $n_c = 3$: $\kappa = 4 > k_{\max} = 3$, so $\lambda_{\max} \to 3$, $t = 3\pi/16$, $C = \frac{4}{3}$; clock values $0.75, 1.5, 2.25, 3$ are rounded: fidelity 0.9532157640.
- $n_c \geq 4$: every k_j is an integer, fidelity 1.

Finding in the notebook

For $n_c = 3$ the notebook prints an "ideal" $P_{\text{success}} = 0.632716$ from $\sum_j |\beta_j|^2 C^2/\lambda_j^2$. With $C = \frac{4}{3} > \lambda_{\min}$ this formula is not a probability ($C/\lambda_1 > 1$); the circuit gives 0.3285.

Shot noise of the read-out



20 repeated samplings per shot count, notebook circuit (re-executed).

- Only accepted shots ($\approx 35.6\,\%$) carry information about $|x\rangle$.
- Error of each estimated $|x_i|^2$ scales as $1/\sqrt{P_{\text{success}} \cdot S}$.
- 20 000 shots $\rightarrow \approx 7100$ accepted $\rightarrow \pm 0.012$.
- Halving the error costs $4\times$ the shots — the read-out cost behind "d/ε^2".

CPU and GPU simulation

SILICO FELLER

choice	CUDA-Q target	note
`cpu + statevector`	`qpp-cpu`	fp64
`cpu + density_matrix`	`density-matrix-cpu`	4^N memory
`gpu + statevector`	`nvidia`	fp32/fp64
`multi-gpu + statevector`	`nvidia (mgpu)`	needs MPI
`hardware`	provider	**refused by HHL.ipynb**

`BACKEND = "auto"` uses a GPU if CUDA-Q sees one, otherwise the CPU (the recorded run: CPU).

- State vector: $16 \cdot 2^N$ bytes (fp64). $N = 7$: 2 kB — trivial on any machine.
- GPUs pay off only for much larger N; their speed was *not* measured for this material.
- Hardware: the notebook raises a `ValueError` — exact e^{iAt} custom gates and amplitude loading cannot run on a QPU.

Noise and error analysis

SILICO FELLER

Algorithmic errors (present even without noise)

- QPE rounding when $k_j \notin \mathbb{Z}$ (fidelity 0.9532157640 at $n_c = 3$)
- Hamiltonian-simulation (Trotter) error — absent here: exact `expm`
- finite shots: $1/\sqrt{\text{accepted}}$

Hardware noise (general, not simulated here)

- Deep circuit: 4-controlled rotations and controlled e^{iAt2^j} decompose into many CNOTs.
- Gate errors corrupt the clock, so uncomputation fails; post-selection then accepts wrong shots.
- Practical HHL needs error correction.

`SIMULATION_METHOD = "density_matrix"` is allowed, but the notebook defines **no noise model**.

Limitations

SILICO FELLER

- **State preparation** of a generic $|b\rangle$: $O(d)$.
- **Hamiltonian simulation** needs sparse or structured A.
- **Conditioning**: cost $\propto \kappa^2$ (or κ with better methods).
- **Output**: a quantum state; the full x needs $O(d/\varepsilon^2)$ shots.
- **Depth**: far beyond today's noisy devices.

Notebook-specific

- exact `expm` gates (no Hamiltonian simulation)
- amplitude loading via `qalloc(vector)` (no state-preparation circuit)
- lookup-table inversion: $2^{n_c} - 1$ rotations
- full-matrix storage (sparsity not used)
- simulator read-out of amplitudes and norm

Practical applications (research directions)

SILICO FELLER

- **Differential equations**: finite-difference/element systems; linear ODEs via linear systems.
- **Machine learning**: least squares, quantum SVM, Gaussian processes.
- **Electromagnetic scattering**, fluid dynamics.
- **Network analysis**: effective resistance, Laplacians.

Common requirement

The useful output must be a **global property** of x (an expectation value, an overlap, a norm), A must be sparse and well conditioned, and $|b\rangle$ easy to prepare.

All of these are proposals; none has shown an end-to-end practical quantum advantage.

Advanced quantum linear-system algorithms

algorithm	key improvement
HHL (2009)	$\tilde{O}(\log(d)s^2\kappa^2/\varepsilon)$ with QPE + controlled rotation
Ambainis (2012)	variable-time amplitude amplification: $\kappa^2 \to \kappa$ (up to logs)
Childs–Kothari–Somma (2017)	LCU / Fourier–Chebyshev series instead of QPE: $\mathrm{poly}\log(1/\varepsilon)$
QSVT (Gilyén et al. 2019)	polynomial approximation of $1/x$ applied to a block-encoding; unifies the above
Adiabatic / randomized (Subaşı–Somma–Orsucci 2019), discrete adiabatic (Costa et al. 2022)	optimal $O(\kappa \log(1/\varepsilon))$ queries
Variational (VQLS, Bravo-Prieto et al.)	short circuits + classical optimiser; heuristic, no guarantee

None of these is implemented in `HHL.ipynb`; they share HHL's input/output caveats.

Try it yourself: `HHL.ipynb`

SILICO FELLER

1. Install CUDA-Q (`pip install cudaq`), open `HHL.ipynb`, run all cells.
2. Edit only Section 3:
   - `NUM_CLOCK_QUBITS = 3`: rounding, fidelity < 1
   - `MATRIX_A = [[1, -1/3], [-1/3, 1]]`, `VECTOR_B = [1, 0]`

Experiments

- `MATRIX_A = "random"`, `RANDOM_CONDITION_NUMBER = 20`, `NUM_CLOCK_QUBITS = 6`
- `MATRIX_A = [[1, 2], [0, 1]]`: automatic Hermitian embedding
- `SHOTS = 2000`: larger read-out error

For $n_c = 3$, compare the printed "ideal" P_{success} with the state value (see the precision slide).

Common mistakes and how to spot them

SILICO FELLER

- $|x\rangle$ **is not** x: it is normalised; the norm needs $\|b\|/C$ (simulator) or P_{success}.
- **Amplitude vs. probability**: C/λ is an amplitude; $P = C^2/\lambda^2$.
- **Negative eigenvalues**: register $k \geq 2^{n_c-1}$ means $k - 2^{n_c}$.
- $C > |\lambda|_{\min}$: arcsin argument above 1 — clipping and wrong "ideal" formulas.
- **Skipping uncomputation**: clock stays entangled, the system is a mixture.
- **Bit order**: `clock[0]` is the MSB; CUDA-Q state vectors index qubit 0 as the least significant bit.
- **Ignoring hidden costs**: `expm` and `qalloc(vector)` are simulator shortcuts.
- **"Exponential speed-up"** for printing all of x: false.

Exercises

1. Condition number of $\left(\begin{smallmatrix} 3 & 1 \\ 1 & 3 \end{smallmatrix}\right)$.
2. Solve $\left(\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right) x = (1, 0)$ through the eigen-decomposition.
3. $n_c = 3$, $t = \pi/4$: clock values of $\lambda = 2$ and of $\lambda = -1$.
4. $n_c = 4$, $C = 1$: the clock reads 1101. Which $\tilde{\lambda}$, and which ancilla amplitude?
5. $C = 1$, $\lambda = 4$: rotation angle and probability of ancilla $= 1$.
6. $A = \mathrm{diag}(1, 2)$, $|b\rangle = \frac{1}{\sqrt{2}}(1, 1)$, $C = 1$: P_{success} and $|x\rangle$.
7. Qubits, rotations and gates of the notebook's circuit for the 4×4 example with $n_c = 5$.
8. Default run: expected accepted shots out of 20 000 and expected repetitions per success.

Answers

SILICO FELLER

1. Eigenvalues 4 and 2: $\kappa = 2$.
2. $\lambda = 3$, $u = \frac{1}{\sqrt{2}}(1,1)$; $\lambda = 1$, $u = \frac{1}{\sqrt{2}}(1,-1)$; $\beta = \frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}$: $x = \frac{1}{6}(1,1) + \frac{1}{2}(1,-1) = \left(\frac{2}{3}, -\frac{1}{3}\right)$.
3. $k = \lambda \cdot \frac{t\,2^{n_c}}{2\pi} = \lambda \cdot \frac{(\pi/4)\cdot 8}{2\pi} = \lambda$: $\lambda = 2 \to k = 2$ (010); $\lambda = -1 \to -1 + 8 = 7$ (111).
4. $13 - 16 = -3$: $\tilde{\lambda} = -3$, amplitude $C/\tilde{\lambda} = -\frac{1}{3}$.
5. $\theta = 2 \arcsin \frac{1}{4} = 0.5054$ rad; $P = \frac{1}{16} = 0.0625$.
6. $P = \frac{1}{2}(1 + \frac{1}{4}) = 0.625$; $x \propto (1, \frac{1}{2})$, $|x\rangle = \frac{1}{\sqrt{5}}(2,1)$.
7. $5 + 2 + 1 = 8$ qubits, $2^5 - 1 = 31$ rotations, 235 gates (`estimate_resources`).
8. $0.355903 \times 20\,000 \approx 7118$ (recorded 7086); $1/0.3559 \approx 2.8$ repetitions.

Summary

SILICO FELLER

- $A^{-1}|b\rangle = \sum_j \frac{\beta_j}{\lambda_j}|u_j\rangle$: inversion is division in the eigenbasis.
- **QPE** with $U = e^{iAt}$ writes λ_j/C into the clock; $R_Y(2\arcsin(C/\tilde{\lambda}))$ puts C/λ_j on the ancilla.
- **Uncompute** the clock, **post-select** ancilla $= 1$: $|x\rangle$ with $P_{\text{success}} = C^2\|A^{-1}|b\rangle\|^2$.
- Cost $\tilde{O}(\log(d)s^2\kappa^2/\varepsilon)$ — only under strong input/output assumptions.
- `HHL.ipynb`: 7 qubits, 111 gates; recovers $x = (25, 7, 11, 5)/48$ exactly (fidelity 1), $P_{\text{success}} = 0.3559$ — reproduced here with CUDA-Q.

Further reading

- A. Harrow, A. Hassidim, S. Lloyd, *Quantum algorithm for linear systems of equations*, PRL 103 (2009)
- S. Aaronson, *Read the fine print*, Nature Physics 11 (2015)
- A. Childs, R. Kothari, R. Somma, SIAM J. Comput. 46 (2017)
- A. Gilyén et al., *Quantum singular value transformation*, STOC (2019)
- NVIDIA CUDA-Q documentation
- This repository: `HHL.ipynb`, `assessments/HHL_*.pdf`